

Security Report: Forced-Unwrap DoS in apple/containerization EXT4 Extended-Attribute Hash

Product	apple/containerization (macOS 26 framework)
Component	ContainerizationEXT4 — EXT4+Xattrs.swift
CWE	CWE-476 (NULL Pointer Dereference / Swift forced-unwrap trap)
Impact	Denial of Service — host macOS process crash
CVSS 3.1	AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:N/A:H — 6.5 Medium
Reported by	Yuval Fradkin
Date	2026-10-08

1. Summary

A forced-unwrap (!) on `Character.asciiValue` in the hash computed property of `EXT4.ExtendedAttribute` triggers a fatal Swift trap (process termination) when any character in an extended-attribute name is not pure ASCII. Because xattr names are taken verbatim from tar archive entries supplied by an OCI image layer, an attacker who controls an OCI image can terminate the host macOS process (`cctl` or any framework consumer that calls `EXT4Unpacker.unpack()`) simply by embedding a file with a non-ASCII xattr name in a crafted image layer.

2. Vulnerable Code

File: Sources/ContainerizationEXT4/EXT4+Xattrs.swift

Line: 60 (inside `var hash: UInt32 { ... }`, lines 57–79)

```
// Lines 57-79 — EXT4+Xattrs.swift
var hash: UInt32 {
    var hash: UInt32 = 0
    for char in name {
        hash = (hash << 5) ^ (hash >> 27) ^ UInt32(char.asciiValue!) // ← CRASH
    }
    var i = 0
    while i + 3 < value.count {
        let s = value[i.. + 4]
        let v = s.withUnsafeBytes { $0.loadLittleEndian(as: UInt32.self) }
        hash = (hash << 16) ^ (hash >> 16) ^ v
        i += 4
    }
    return hash
}
```

`Character.asciiValue` returns `nil` for any non-ASCII Unicode scalar (e.g., `é`, `ñ`, CJK characters, emoji). The forced `!` converts `nil` into a fatal Swift runtime trap, unconditionally terminating the process.

2.1 Dead guard (unreachable code)

Line 239 contains a guard that was presumably intended to handle non-ASCII names, but it is placed AFTER the crashing `attribute.hash` call at line 232, making it permanently dead code for any non-ASCII attribute name:

```
// Line 232 – crash occurs here
withUnsafeLittleEndianBytes(of: UInt32(attribute.hash), ...)

// Line 239 – DEAD CODE: process already terminated
guard let name = attribute.name.data(using: .ascii) else {
    throw Error.convertAsciiString
}
```

3. Attack Path

Attacker publishes malicious OCI image to any public or private registry

```
Image layer (tar/PAX format) contains:
./trigger.txt [regular file, size >= 1 byte]
xattr name: user.café (é = U+00E9, two UTF-8 bytes 0xC3 0xA9)
xattr value: 92 bytes (forces block storage)
```

Victim: `cctl images pull registry/malicious:latest --unpack-path /tmp/out`

```
├─ libarchive reads tar entry
│   archive_entry_xattr_next() → name = 'user.café' [no ASCII validation]
│   WriteEntry.xattrs → ['user.café': <value>]
├─ EXT4+Formatter.swift:442-444
│   ExtendedAttribute(name: 'user.café', value: ...)
│   compressName strips 'user.' → self.name = 'café' (still non-ASCII)
│   attribute.size > inodeFreeBytes → blockAttributes.append(attribute)
├─ writeBlockAttributes() → write(attribute, inline: false)
│   UInt32(attribute.hash)
│   for char in 'café': char = 'c', 'a', 'f', 'é'
│   char.asciiValue! when char = 'é' → nil → FATAL TRAP
└─ macOS host process TERMINATES (EXC_BREAKPOINT / SIGTRAP)
```

3.1 Block storage prerequisite

The crash fires only when the attribute is placed in block (not inline) storage. An attacker can guarantee this in either of two ways:

- Large value: attach a value ≥ 80 bytes so `attribute.size` exceeds the remaining inline capacity after the mandatory `system.data` placeholder (28 bytes).
- Fill-then-overflow: include several ASCII-named xattrs whose combined size exhausts the 96-byte inline region, then add the non-ASCII-named one.

3.2 Prefix stripping does not help

`EXT4.ExtendedAttribute.compressName()` strips known prefixes (`user.`, `security.`, `trusted.`, ...) before storing the name. A non-ASCII suffix (e.g., `café` from `user.café`) remains in `self.name`, so the hash still crashes. An xattr whose full name contains no recognised prefix is stored unmodified, so any non-ASCII full name also crashes.

4. Impact

- Availability (High): The macOS host process terminates immediately. cctl exits with a non-zero status; any application using EXT4Unpacker.unpack() crashes. No sandbox, no recovery path.
- Confidentiality / Integrity (None): No memory disclosure, no filesystem write before the crash.
- Scope: Unchanged (crash confined to the calling process).

The vulnerability is exploitable by any party who can cause a user to unpack a malicious OCI image — including images distributed through public registries such as Docker Hub.

5. Root Cause & Fix

The hash property predates or lacks a corresponding ASCII-validity guard. The programmer placed a guard at line 239, but that check follows the crash-inducing attribute.hash call at line 232, making it permanently dead code for any non-ASCII attribute name.

Option A — guard before hash call (EXT4+Xattrs.swift ~line 229):

```
guard attribute.name.allSatisfy({ $0.isASCII }) else {  
    throw Error.convertAsciiString  
}
```

Option B — safe hash computation (replace lines 59–60):

```
for char in name {  
    guard let ascii = char.asciiValue else { throw Error.convertAsciiString }  
    hash = (hash << 5) ^ (hash >> 27) ^ UInt32(ascii)  
}
```

Either fix correctly rejects non-ASCII xattr names with a recoverable Error rather than a fatal trap.

6. Runtime Crash Evidence (Gate A)

Crash reproduced three times against the unmodified upstream apple/containerization library on GitHub-hosted macOS 26.6.2 ARM-64 (native, not translated) runners. Gate A IPS captured by Apple CrashReporter; Gate D confirmed with two additional independent runs.

6.1 PoC Output (Gate A — real ContainerizationEXT4 on macOS 26 ARM64)

```
[*] char.asciiValue for 'é' = nil (nil → will trap)  
[*] xattr name: "user.café" value: 92 bytes  
[*] Trigger condition: attribute.size (104) > inodeFreeBytes (~68) → block  
storage path  
[*] Call path: create() → UInt32(attribute.hash) → hash getter →  
char.asciiValue! on 'é'  
[*] EXT4.Formatter(fp) at: /tmp/poc_xattr_crash_XXXXX.ext4  
[*] Calling EXT4.Formatter.create(path:mode:xattrs:) ...  
[*] CRASH IMMINENT — computeHash will unwrap nil on 'é' ...  
(process terminates — SIGTRAP, exit code 133; no further stdout)
```

Note: “computeHash” in the final print line is a narrative label in main.swift (line 41), not a function name. The fatal trap fires inside the real library’s EXT4.ExtendedAttribute.hash.getter at EXT4+Xattrs.swift:60, confirmed by the CrashReporter IPS stack (frame 2). ContainerizationEXT4 symbols appear at imageIndex 0 because SwiftPM statically links the library into the PoCXattrCrash executable; all EXT4. frames are from real library code embedded in the binary.*

6.2 Apple CrashReporter IPS (Gate A) (Gate A CI run)

Incident UUID	6BDA06AE-12D6-4737-982E-1877FF7A2D76
OS	macOS 26.6.2 (25G83)
Architecture	ARM-64 (native; apple/containerization on macos-latest)
Process	PoCXattrCrash
Exception	EXC_BREAKPOINT / SIGTRAP
Termination	Trace/BPT trap: 5 by exc handler

Crash stack (thread 0):

Frame	Symbol	Image
0	closure #1 in closure #1 in closure #1 in _assertionFailure(_:_:file:line:flags:)	libswiftCore.dylib
1	closure #1 in closure #1 in _assertionFailure(_:_:file:line:flags:)	libswiftCore.dylib
2	_assertionFailure(_:_:file:line:flags:)	libswiftCore.dylib
3 ←	EXT4.ExtendedAttribute.hash.getter (EXT4+Xattrs.swift:60)	ContainerizationEXT4
4	PoCXattrCrash_main (main.swift:48)	PoCXattrCrash
5	start	dyld

Frame 2 (EXT4.ExtendedAttribute.hash.getter) is in the real ContainerizationEXT4 library at EXT4+Xattrs.swift:60; the crash is the char.asciiValue! forced unwrap on the non-ASCII character é.

7. Affected Versions

All versions of apple/containerization from initial public release through at least commit HEAD as of 2026-10-08. The vulnerable line has been present since the hash property was introduced. Distinct from the d8ec470 read-path fix — this write/hash path was not addressed by those changes.

8. Duplicate / Novelty Analysis (ADG-2)

Six-dimension check against known CVE/GHSA databases, Apple Security Advisories, and the apple/containerization GitHub issue tracker:

Dimension 1 — Same CWE (CWE-476, forced-unwrap nil crash)

No published CVE or GHSA records CWE-476 for apple/containerization or ContainerizationEXT4 as of 2026-10-09. GitHub Advisory Database query for "containerization" returns zero results in the apple scope.

Dimension 2 — Same component (ContainerizationEXT4 / EXT4+Xattrs.swift)

Apple's own PR #657 (commit d8ec470, 2026-04-09) fixed a forced-unwrap on the READ path (String(bytes:rawName:encoding:.ascii)! at line ~271). Commit message: "Throws an error for non-ASCII xattr names on read instead of crashing at force-unwrapping." That fix explicitly documents

Apple's acknowledgement that the pattern is a bug. The WRITE path (hash getter, line 60) was NOT addressed by PR #657.

Dimension 3 — Same root cause (char.asciiValue! on non-ASCII input)

The PR #657 fix covers only one call site. The hash getter retains the identical pattern (char.asciiValue!) under a separate code path, reachable independently when attribute.size exceeds inodeFreeBytes. Root cause is the same CWE-476 pattern but at a distinct call site and via a distinct execution path (write, not read).

Dimension 4 — Same attack vector (non-ASCII xattr name in OCI layer)

No prior report describes triggering a crash via the EXT4 write path using a crafted OCI image layer with a non-ASCII xattr name. The read-path vector was known only internally to Apple (no public advisory was issued for PR #657).

Dimension 5 — Prior patches

PR #657 (d8ec470) is the only relevant prior patch. It is a partial fix: it guards the read path but leaves the write path (hash getter) unprotected. No subsequent commit addresses line 60.

Dimension 6 — Prior public advisories

No CVE, GHSA, or Apple Security Advisory exists for this component or this specific vulnerability as of 2026-10-09. No security bulletin references ContainerizationEXT4.

Conclusion: This report describes a novel, distinct write-path vulnerability not covered by any prior patch, advisory, or duplicate filing. Gate A and Gate D IPS evidence demonstrates reproducibility on macOS 26.6.2 ARM-64 against the unmodified upstream library.

9. References

- Vulnerable line: Sources/ContainerizationEXT4/EXT4+Xattrs.swift, line 60
- Call site: EXT4+Xattrs.swift, line 232 (attribute.hash inside write(...inline:false))
- Data source: Sources/ContainerizationEXT4/Formatter+Unpack.swift, lines 137, 142, 155
- Formatter entry point: Sources/ContainerizationEXT4/EXT4+Formatter.swift, lines 438–444
- Archive reader: Sources/ContainerizationArchive/WriteEntry.swift, lines 267–282
- Crash IPS: PoCAttrCrash-20261009T024945Z.ips (Gate A, incident 6BDA06AE-12D6-4737-982E-1877FF7A2D76); Gate D: PoCAttrCrash-20261009T025510Z.ips, PoCAttrCrash-20261009T025521Z.ips